SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

Name: Hemalakshmi H
Reg.No.: 192571049

Assessment Report: Distributed AI System Design Using PySpark RDDs and Intelligent Agents

Introduction

Modern AI applications require scalable, distributed systems capable of processing massive datasets in real time. PySpark, with its Resilient Distributed Datasets (RDDs), provides a powerful foundation for building such systems. Intelligent agents further enhance adaptability, decision-making, and automation. This report presents the design of five advanced AI systems—each integrating PySpark RDD-based distributed processing with agent-driven intelligence. The designs emphasize architecture, data flow, scalability, and real-time adaptation.

1. Hybrid AI Recommendation Engine Design

Objective

To design a hybrid recommendation engine combining **content-based filtering** and **collaborative filtering**, implemented using PySpark RDDs and enhanced by intelligent agents for real-time personalization.

System Architecture

- **Data Sources:** User ratings, browsing history, item metadata, tags, categories.

- **Distributed Storage:** HDFS storing user–item interactions and item profiles.
- **Processing Layer:** PySpark RDDs for similarity computation, feature extraction, and score generation.
- **Agent Layer:** Adaptive agents monitoring user behavior and updating recommendation weights.
- **Output Layer:** Personalized ranked list of items.

Role of PySpark RDDs

- User–item interactions stored as RDDs: (user_id, item_id, rating)
- Item features stored as RDDs: (item_id, feature_vector)
- Collaborative filtering via RDD operations:
    - map, join, reduceByKey for similarity matrices
- Content-based filtering via vector similarity computations
- Hybrid score = weighted sum of both models, computed in parallel.

Intelligent Agent Behavior

- Tracks user clicks, skips, dwell time, and feedback.
- Adjusts weights between content-based and collaborative filtering.
- Performs exploration–exploitation to introduce new items.
- Learns from user feedback to refine recommendations.

Outcome

A scalable, adaptive recommendation engine capable of handling millions of interactions while delivering personalized results.

2. Smart Disaster Detection System

Objective

To design a distributed AI system that detects natural disasters using multi-source data streams processed through PySpark RDDs and coordinated by intelligent agents.

System Architecture

- **Data Sources:**
    - Satellite imagery
    - IoT sensors (water level, seismic activity)
    - Social media feeds
- **Data Ingestion:** Kafka streams → PySpark streaming.
- **Processing Layer:**
    - Image feature extraction
    - Sensor anomaly detection
    - Social media text analysis
- **Fusion Layer:** RDD-based spatial–temporal fusion.
- **Agent Layer:** Early warning agents, response agents, coordination agents.
- **Output Layer:** Alerts, risk maps, response strategies.

Data Fusion Techniques

- Satellite tiles mapped as RDDs: (region, image_features)
- Sensor readings: (sensor_id, timestamp, value)
- Social media: (geo_tag, text_features)
- Fusion via:
    - join on region
    - groupByKey on time windows
    - Weighted risk scoring

Intelligent Agent Coordination

- **Detection agents:** Identify anomalies across fused data.
- **Warning agents:** Trigger alerts when risk thresholds exceed limits.
- **Response agents:** Suggest evacuation routes, resource allocation.
- **Learning agents:** Improve thresholds using historical outcomes.

Outcome

A real-time, distributed disaster detection system capable of early warnings and coordinated response.

3. AI-Based Fraud Detection Framework

Objective

To design a scalable fraud detection system using distributed anomaly detection algorithms implemented with PySpark RDDs and intelligent agents.

System Architecture

- **Data Sources:** Financial transactions, user profiles, merchant data.
- **Processing Layer:** RDD-based anomaly detection.
- **Agent Layer:** Fraud detection agents, alert agents, learning agents.
- **Output Layer:** Fraud alerts, risk scores, flagged transactions.

Distributed Anomaly Detection

- Transactions stored as RDDs: (txn_id, user_id, amount, location, time, merchant, features)
- Statistical profiling using:
    - reduceByKey for mean/variance
- Outlier detection using:
    - Z-score
    - Clustering (distributed k-means)
- Pattern detection using:
    - Time-based grouping
    - Location-based anomaly checks

Intelligent Agent Collaboration

- **Detection agents:** Identify suspicious transactions.
- **Alert agents:** Notify banks or block transactions.
- **Feedback agents:** Learn from confirmed fraud cases.
- **Pattern agents:** Share fraud patterns across regions.

Outcome

A continuously improving fraud detection system capable of processing millions of transactions per minute.

4. Autonomous Supply Chain Optimization System

Objective

To design a distributed AI system that optimizes supply chain operations using PySpark RDDs and intelligent agents.

System Architecture

- **Data Sources:** Inventory levels, demand history, shipment data, supplier details.
- **Processing Layer:** RDD-based forecasting, cost analysis, route optimization.
- **Agent Layer:** Inventory agents, logistics agents, demand agents.
- **Output Layer:** Optimized routes, reorder decisions, demand forecasts.

Role of RDD Transformations

- Inventory data: (warehouse, product, stock)
- Demand history: (product, timestamp, demand)
- Logistics routes: (route, cost, time)
- RDD operations:
    - map for feature extraction
    - filter for constraints

    - join for combining datasets
    - aggregateByKey for forecasting

Intelligent Agent Interaction

- **Inventory agents:** Decide reorder quantities.
- **Logistics agents:** Select optimal routes.
- **Demand agents:** Update forecasts based on market signals.
- **Coordinator agents:** Ensure decentralized decisions align with global optimization.

Outcome
A self-optimizing supply chain system capable of reducing cost and improving efficiency.

5. Personalized Learning Analytics Platform

Objective

To design an AI-driven educational platform using PySpark RDDs and intelligent agents for large-scale personalized learning.

System Architecture

- **Data Sources:** Student activity logs, quiz scores, content usage patterns.
- **Processing Layer:** RDD-based analytics and prediction models.
- **Agent Layer:** Learning agents, feedback agents, personalization agents.
- **Output Layer:** Personalized content, performance insights, adaptive learning paths.

Scalable Analytics Using RDDs

- Student interactions: (student_id, activity_type, resource_id, timestamp, score)
- RDD operations:
    - groupByKey for engagement metrics
    - reduceByKey for performance trends
    - Windowed analysis for learning pace

- Predictive models for:
    - Dropout risk
    - Mastery level
    - Content recommendation

Intelligent Agent Support

- **Learning agents:** Adjust difficulty and content sequence.
- **Feedback agents:** Provide hints, remedial content, challenges.
- **Monitoring agents:** Track real-time performance.
- **Adaptive agents:** Continuously refine learning paths.

Outcome

A scalable, adaptive learning platform capable of personalizing education for thousands of learners.

Conclusion

This assessment demonstrates the design of five advanced distributed AI systems integrating PySpark RDDs with intelligent agents. Each system leverages distributed data processing for scalability and agent-based intelligence for adaptability. These designs reflect modern AI engineering practices suitable for large-scale, real-time applications across domains such as recommendation systems, disaster management, fraud detection, supply chain optimization, and personalized learning.